Data Session 14, part 1: The Finest Grain

84-355 Democracy's Data | Precinct returns, and why they are hard to get

Jonathan Cervas

August 10, 2026

Everything here runs as given. Your job is to read what comes out and say what it means.

Where did that number come from?

> *"That precinct went 71% for Biden."*

This is the last data session, and it uses the smallest unit there is. A **precinct** is the building block of American elections — a few hundred to a few thousand voters, the level at which ballots are actually counted before anything is added up.

Everything you have done this term has been coarser. County returns in Session 1. State totals. Congressional districts. **Precincts are where the count happens**, and they are the only geography fine enough to sit next to census blocks — which is what the second half of today needs.

So: go and get Georgia's precinct results.

Part 1 — Four things stand in your way

One. The counties hold the record, not the state. Georgia's precinct boundaries and results are maintained by each of its **159 counties** separately. Houston County posts its Statement of Votes Cast on its own website — as a PDF. There are 159 such websites, each with its own naming and format, and no county-level route that scales.

Two. The state does compile it — twice, in two different places. The Secretary of State publishes a statewide archive, one ZIP per election back to 2012, containing every county's own tabulation export in XML, TXT and XLS. It is the primary source: the counties' machine-readable output, not somebody's reconstruction of it. That is what this lab reads.

Three. And it is behind a bot check. The URLs are stable and public, and a browser fetches them without complaint. A script gets `403` — Cloudflare protection, verified with browser headers and a referer:

```
HTTP/2 403   (identical URL, identical headers, from a browser: 200)
```

Four. So for years the file everyone actually used came from volunteers. VEST — the Voting and Election Science Team — reads county returns, reconciles them, matches precincts to boundaries and publishes clean files for the whole country. Excellent work, and an academic project. The archive hosting

it asks for a **guestbook response** before handing a file over, which a person does in thirty seconds and a program cannot do at all.

This lab used to depend on that. It does not any more: going upstream to the Secretary of State's own publication got the same data from the primary source — and got something VEST does not carry, which is *how* each ballot was cast. You will see those four labels in Part 2.

The finest-grained public record of American voting is held by 159 county governments, compiled by the state behind a bot check, and reconstructed by volunteers behind a form. Hold that next to Session 13, where Alaska published every individual ballot as machine-readable JSON because it decided to.

Part 2 — Reading a column name

The file arrived without a codebook. Everything you need in order to use it is carried in **names** — the names of the columns, the name of the contest, the names of the four ways a ballot can be cast — and names are written by people, one county at a time.

Start with the inventory of what the source actually contains.

```
st <- read.csv("data/structure.csv", stringsAsFactors = FALSE)
print(st[st$item != "vote methods", ], row.names = FALSE)
```

```
#>                                  item                          value
#>                               counties                            159
#>                              precincts                           2656
#>                                contest President of the United States
#>                             candidates                              3
#>  contests in the source (median county)                             23
#>                rows in vote_methods.csv                          31872
```

```
# the one value too wide for that table: how the ballot was cast
strsplit(st$value[st$item == "vote methods"], " \\| ")[[1]]
```

```
#> [1] "Absentee by Mail Votes" "Advanced Voting Votes"  "Election Day Votes"
#> [4] "Provisional Votes"
```

The presidential race is **one contest among the many** the median county reports, and every vote in it is recorded three ways at once: by precinct, by candidate, and by method. That last one is what going upstream bought.

The columns are candidates

```
n20 <- names(read.csv("data/ga2020_precincts.csv", nrows = 1, check.names = FALSE))
n24 <- names(read.csv("data/ga2024_precincts.csv", nrows = 1, check.names = FALSE))
n20
```

```
#> [1] "county"            "precinct"          "registered"
#> [4] "ballots_cast"      "Joseph R. Biden"   "Donald J. Trump"
#> [7] "Jo Jorgensen"      "total"             "dem_two_party_pct"
```

```
n24
```

```
#>  [1] "county"             "precinct"           "registered"
#>  [4] "ballots_cast"       "Donald J. Trump"    "Kamala D. Harris"
#>  [7] "Chase Oliver"       "Jill Stein"         "Claudia De la Cruz"
#> [10] "Cornel West"        "total"              "dem_two_party_pct"
```

```
intersect(n20, n24)
```

```
#> [1] "county"            "precinct"          "registered"
#> [4] "ballots_cast"      "Donald J. Trump"   "total"
#> [7] "dem_two_party_pct"
```

Nothing in either file says which column is a Republican. The party is in your head, not in the data. Now look at what the two files share: the structural columns — county, precinct, registration, totals — and **exactly one candidate**, because exactly one candidate was on both ballots.

That is the dangerous case, not the safe one. Code written for 2020 that asks for `Jo Jorgensen` fails loudly against the 2024 file, and loud failures are cheap. Code that asks for `Donald J. Trump` runs against both and returns two different elections, and code that reaches for "the fifth column" gets Trump in one file and Biden in the other without a word of complaint.

The same office, three names

```
vm  <- read.csv("data/ga2020_vote_methods.csv", stringsAsFactors = FALSE)
s24 <- read.csv("data/ga2024_structure.csv", stringsAsFactors = FALSE)

table(vm$contest)                          # 2020: two labels (rows, not votes)
```

```
#>
#>                                President of the United States
#>                                                         30000
#> President of the United States/Presidentede los Estados Unidos
#>                                                          1872
```

```
s24$value[s24$item == "contest"]                    # 2024: a third
```

```
#> [1] "President of the US"
```

```
unique(vm$county[grepl("Presidente", vm$contest)])  # who writes the long one
```

```
#> [1] "Gwinnett"
```

One election, one office, and the label is not the same twice. Georgia's bilingual counties name the contest in both languages, and four years later the newer publishing platform shortened *United States* to *US* — a difference that means nothing to a reader and everything to a match.

The contest label is a sentence in a natural language, and it is the only key you have. Here is what that cost, in the words of the script that parses this folder:

```
src <- readLines("data/parse-ga-sos.py", encoding = "UTF-8")
i   <- grep("ANCHOR THE CONTEST MATCH", src)[1]
writeLines(sub("^\\s*# ?", "", src[i:(i + 10)]))
```

```
#> ANCHOR THE CONTEST MATCH. An earlier version matched the bare substring
#> "President" and silently swept in
#>     "Co Commission Chair/Presidente de la Comisión del Condado"
#> -- County Commission Chair, in Spanish -- which put a Gwinnett commission
#> candidate into the presidential totals with 233,110 votes. Georgia's
#> bilingual counties label the real contest
#>     "President of the United States/Presidentede los Estados Unidos"
#> so the match has to be specific enough to keep that and drop the other.
#> The two sources also name the race differently -- the XML says "President
#> of the United States", the 2024 JSON says "President of the US" -- so the
#> default is the longest prefix common to both.
```

A county-commission race, labelled in Spanish, matched the word *President* and was added to the presidential totals. Nothing errored. The file was complete, the arithmetic was right, and the number was wrong by the entire size of a Gwinnett County commission contest.

The methods drift the same way

```
m20 <- strsplit(st$value[st$item == "vote methods"], " \\| ")[[1]]
m24 <- strsplit(s24$value[s24$item == "vote methods"], " \\| ")[[1]]

data.frame(label_2020       = m20,
           same_in_2024     = m20 %in% m24,
           same_minus_Votes = sub(" Votes$", "", m20) %in% m24)
```

```
#>               label_2020 same_in_2024 same_minus_Votes
#> 1 Absentee by Mail Votes        FALSE             TRUE
#> 2  Advanced Voting Votes        FALSE             TRUE
#> 3     Election Day Votes        FALSE             TRUE
#> 4      Provisional Votes        FALSE             TRUE
```

```
m24
```

```
#> [1] "(not reported)"   "Absentee by Mail" "Advanced Voting"  "Election Day"
#> [5] "Provisional"
```

Not one of the four labels survives verbatim. Drop the word **"Votes"** — a word that carries no information at all, in a file that is entirely votes — and all four match. Whether a join between the two elections works turns on that word.

The 2024 list also has a fifth entry that is not a way of voting. `(not reported)` is bookkeeping: where the source gives a precinct's total but no breakdown, the parser books the remainder there rather than dropping it. **A label describing the file sits in the same column as four labels describing the election**, and only the header of `data/parse-ga-sos.py` tells you which is which.

You cannot use this file without being told all of that. None of it is in the data. A student who downloads it and starts computing will produce something confidently wrong, and — this is the part worth keeping — it will not look wrong. Every session this term has had a version of this problem; this is the most compressed one.

For contrast: how VEST solved it

Until it moved upstream, this lab read the volunteer file instead — and VEST does not have the problem above, because it does not use names. It puts the codebook **inside the column name**:

```
G20PRERTRU
| | | | |
| | | | +--- TRU  first three letters of the surname
| | | +----- R    party
| | +------- PRE  office
| +--------- 20   year
+----------- G    general election (P primary, R runoff, S special)
```

This lab no longer reads that file, and the comparison is the point. A code like `G20PRERTRU` cannot drift, because nobody edits it and no county writes it in Spanish. What it cannot do is explain itself: it is unusable without a document that does not ship with the data, and the failure it produces is a person confidently misreading a column rather than a program silently matching the wrong one.

Two designs, two failure modes, and neither file is usable by somebody who was not told. That is the more useful version of the lesson, and it is why the codebook question is worth arguing about rather than answering.

Part 3 — What a precinct is, in practice

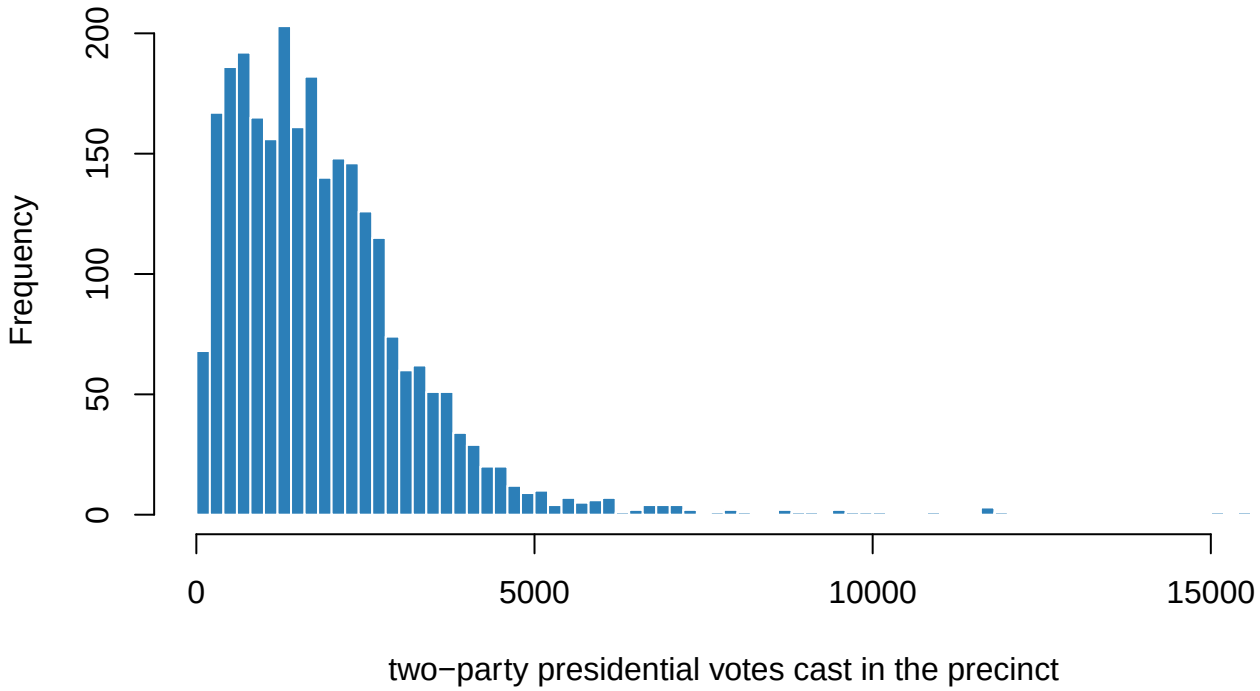
```
p  <- read.csv("data/precincts.csv", stringsAsFactors = FALSE)
co <- read.csv("data/counties.csv",  stringsAsFactors = FALSE)

c(precincts = nrow(p), counties = nrow(co),
  votes = sum(p$total), empty_precincts = sum(p$total == 0))
```

```
#>       precincts        counties           votes empty_precincts
#>            2656             159         4998482               3
```

```
summary(p$total)
```

```
#>    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
#>     0.0   860.8  1628.5  1882.0  2527.2 15531.0
```

```
hist(p$total[p$total > 0], breaks = 60, col = "#2c7fb8", border = "white",
     main = "", xlab = "two-party presidential votes cast in the precinct")
```



"Precinct" is not a standard size. Look at the spread — and then look at where the extremes actually are, because the obvious answer is wrong.

```
np    <- p[p$total > 0, ]
one   <- co$county[co$precincts == 1]   # counties reported as a single precinct
metro <- c("Fulton", "DeKalb", "Gwinnett", "Cobb", "Clayton")

big <- head(np[order(-np$total), c("county", "precinct", "total")], 6)
big$whole_county_is_one_precinct <- big$county %in% one
print(big, row.names = FALSE)
```

```
#>    county               precinct total whole_county_is_one_precinct
#>   Lumpkin              Dahlonega 15531                          TRUE
#>    Coweta          Newnan Centre 15134                         FALSE
#>  Stephens          Senior Center 11885                          TRUE
#>     Butts Butts County Admin Bldg 11771                          TRUE
#>   Jackson         Central Jackson 11757                         FALSE
#>   Jackson            West Jackson 11693                         FALSE
```

```
c(one_precinct_counties        = length(one),
  largest_precinct_statewide   = max(np$total),
  largest_in_the_metro_five    = max(np$total[np$county %in% metro]),
  median_size_metro            = median(np$total[np$county %in% metro]),
  median_size_elsewhere        = median(np$total[!np$county %in% metro]),
  most_precincts_in_one_county = max(co$precincts))
```

```
#>        one_precinct_counties   largest_precinct_statewide
#>                         16.0                      15531.0
#>    largest_in_the_metro_five            median_size_metro
#>                       5509.0                       1887.5
#>        median_size_elsewhere most_precincts_in_one_county
#>                       1453.0                        384.0
```

The biggest precincts in Georgia are rural. Several of the largest are counties that never subdivided at all and report the whole county as one precinct, and the largest precinct anywhere in the metro five — Fulton, DeKalb, Gwinnett, Cobb, Clayton — is a fraction of the state record.

Be careful what that does and does not show, because the seductive reading is also wrong. **Metro precincts are not small**: their median is the higher of the two above. What metro counties do is *cap* the maximum, because a county with hundreds of precincts is managing polling places. So **the maximum is a fact about administration and the median is a fact about population** — two different things are being called "precinct size."

The word therefore means one thing in a county with hundreds of precincts and another in a county with one, and any per-precinct average is averaging over units that are not comparable — the same problem the EAVS session had with "jurisdiction," at a finer scale.

```
# The county file no longer ships a precomputed spread, so derive it here --
# which is better anyway, because you can see how it was derived.
sz  <- tapply(p$total, p$county, function(x) c(min(x), max(x)))
spr <- data.frame(
  county            = names(sz),
  precincts         = as.vector(tapply(p$total, p$county, length)),
  smallest_precinct = vapply(sz, `[`, 1, FUN.VALUE = 0),
  biggest_precinct  = vapply(sz, `[`, 2, FUN.VALUE = 0),
  row.names = NULL, stringsAsFactors = FALSE)
spr$ratio <- round(spr$biggest_precinct / pmax(spr$smallest_precinct, 1))
head(spr[order(-spr$ratio), ], 8)
```

```
#>        county precincts smallest_precinct biggest_precinct ratio
#> 60     Fulton       384                 0             5437  5437
#> 92    Lowndes        13               121            10963    91
#> 45      Dodge        16                73             3181    44
#> 98   McIntosh         6                42             1706    41
#> 34     Coffee         6               253            10086    40
#> 27  Chattooga        13               106             3537    33
#> 121  Richmond        68               111             3590    32
#> 38     Coweta        26               539            15134    28
```

Part 4 — The join nobody warns you about

The reason precincts matter for the rest of today is that racially polarized voting needs **returns and demographics in the same place**. Returns come by precinct. Demographics come by census block.

They do not nest. Census blocks are drawn by the Census Bureau once a decade. Precincts are drawn by counties, change between elections, and are not required to respect block boundaries. So joining them means deciding how to split a block that straddles two precincts — and there is no correct answer, only conventions.

```
c(precincts_in_georgia = nrow(p),
  census_blocks_in_houston_county_alone = 3269)
```

```
#>                  precincts_in_georgia census_blocks_in_houston_county_alone
#>                                  2656                                  3269
```

Houston County — the one county Sessions 7 and 14 use — has **3,269 census blocks**. The state has a few thousand precincts. The units are of completely different orders, they were drawn by different people for different reasons, and **every precinct-level demographic estimate in American politics rests on allocating one to the other.**

That is the machinery under the RPV analysis you do next. It is worth knowing it is there before trusting anything built on top of it.

Part 5 — Your turn

Change **one** thing, re-run, and write about what changed. Pick one.

Option A — The empty precincts. Some precincts recorded zero presidential votes. Find them. Are they real places with no voters, administrative placeholders, or something else? What would you do with them in an analysis, and what would dropping them silently do to a county's numbers?

Option B — Aggregate up. Sum the precincts to counties and compare your totals to `counties.csv`. Then ask the harder question: if precincts aggregate cleanly to counties, why can they not aggregate cleanly to census blocks? The answer is about who drew each boundary and when.

Option C — Size and politics. Is there a relationship between how large a precinct is and how it voted? Plot `dem_two_party_pct` against `total`. If you find one, say what it is really measuring — because it is almost certainly not about precinct size.

Option D — Your own question. Every precinct in Georgia, both elections, and the labels you now know not to trust.

What this lab cannot tell you

- **Whether the count is right.** These are the counties' own tabulation exports, which is the best provenance available and still not a guarantee. Georgia counted the same 2020 ballots a second time and got a slightly different answer; the recount was published at **county** level only, so the finest-grained file in the country is the count that was superseded.
- **What the boundaries look like.** We have returns by precinct name, not the shapes. The shapefiles are a separate download, and matching a name in this file to a polygon in that one is its own problem — the same problem as Part 2, with worse consequences.
- **How 2024 compares, without doing the labels by hand.** 2024 *is* here — `data/ga2024_precincts.csv` and `data/ga2024_counties.csv`, fetched by script rather than waited for. What is not here is a stable key between the two elections: different precincts, different candidates, different labels.
- **How to split a block between two precincts.** The lab names the problem; it does not solve it, and neither does anybody else, cleanly.

On the discussion board

By **Friday, 11:59 p.m.** A sentence or two is enough.

Three that would make good posts:

- For most of the country, the most detailed public record of how Americans voted is still compiled by volunteers from PDFs. Should it be?
- Georgia labels the same office three different ways, and a column called `G20PRERTRU` is unusable without a codebook you have to find separately. Which is the worse design, and whose job is it to fix either one?

- Precincts and census blocks do not line up, so every precinct-level demographic number involves a judgement call. Does that change how you read the RPV result we do next?

Where the data came from

Georgia Secretary of State, *November 3, 2020 General Election* and *November 3rd General Election Recount* — the counties' own tabulation exports, all 159 of them, from the historical archive at https://sos.ga.gov/page/historical-elections-results. The 2024 files come from the newer results platform, as a single JSON export at `results.sos.ga.gov/cdn/results/Georgia/export-2024NovGen.json`.

`data/parse-ga-sos.py` reads both formats and writes the per-precinct, per-county and per-vote-method CSVs; `data/build-data.R` assembles the small derived files this lab reads. `data/README.md` has the download steps. **The raw sources are not committed** — they are large and not ours to redistribute — and the build script prints the instructions and stops rather than failing quietly if they are absent.

The 2020 archive is a manual browser download because of the bot check in Part 1; the 2024 JSON is on a CDN path that is not, and a script fetches it unattended. **The same state, four years apart, gated one way and not the other.**

This lab previously read VEST, *Precinct-Level Returns 2020 by Individual State*, Harvard Dataverse, https://doi.org/10.7910/DVN/NT66Z3, which requires a guestbook response to download. It is cited here as history and as the contrast in Part 2, not as the source of anything in this lab.